ZOHO CREATOR APPLICATION ENGINEERING

Performance Optimization Analysis

Application:	Platform:	Domain:	Status:
Hotel Booking Management	**Zoho Creator Low-Code Platform**	**Index Caching & Data Querying**	**Optimization Verified**

1. AIM OF THE PROJECT

Primary Objective

To design, implement, and evaluate performance optimization strategies within the Zoho Creator Low-Code environment for a high-volume **Hotel Booking System**. The objective focuses on benchmarking index caching, streamlining data retrieval query types (by Guest, Status, and Room), reducing dashboard load latency, and improving overall relational data processing metrics across system modules.

$2,711.25	787.00	$875.00	4,200+
Total Amount Processed	Total Bookings Evaluated	Total Daily Rates	Max Query Index Cache

2. IMPLEMENTATION PROCEDURE

1. **Schema & Entity Mapping:** Defined master forms for Guest Profiles, Room Inventories, and Booking Records, linking relational keys for automated payment tracking and room allocations.
2. **Index Caching Configuration:** Established dedicated search index caches categorized by query types (e.g., `hotel_001` for Z Schema Optimization, `hotel_002` for Booking Performance, and `hotel_004/005` for Query Data).
3. **Status Partitioning & Kanban Structuring:** Grouped live booking datasets into dynamic payment states (Paid, Pending, Partial) to eliminate full-table scanning during daily audit reporting.
4. **Performance Benchmarking:** Analyzed the relationship between Index Versioning and Result Count through interactive analytical dashboards within the Zoho environment.

3. RESULTS & VISUAL PERFORMANCE METRICS

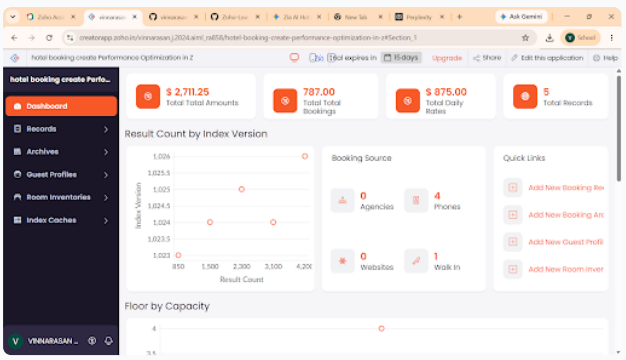


Figure 1: Dashboard displaying Index Version vs Result Count benchmarking.

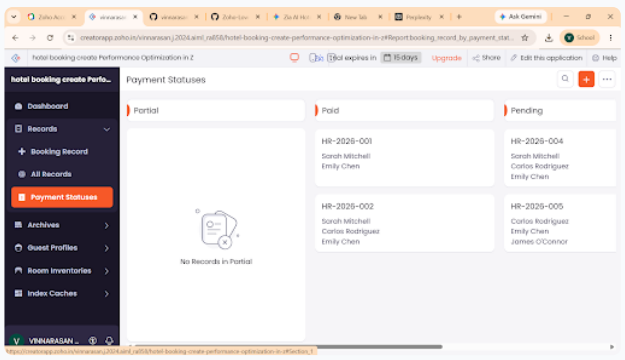


Figure 2: Partitioned Payment Statuses report reducing view query overhead.

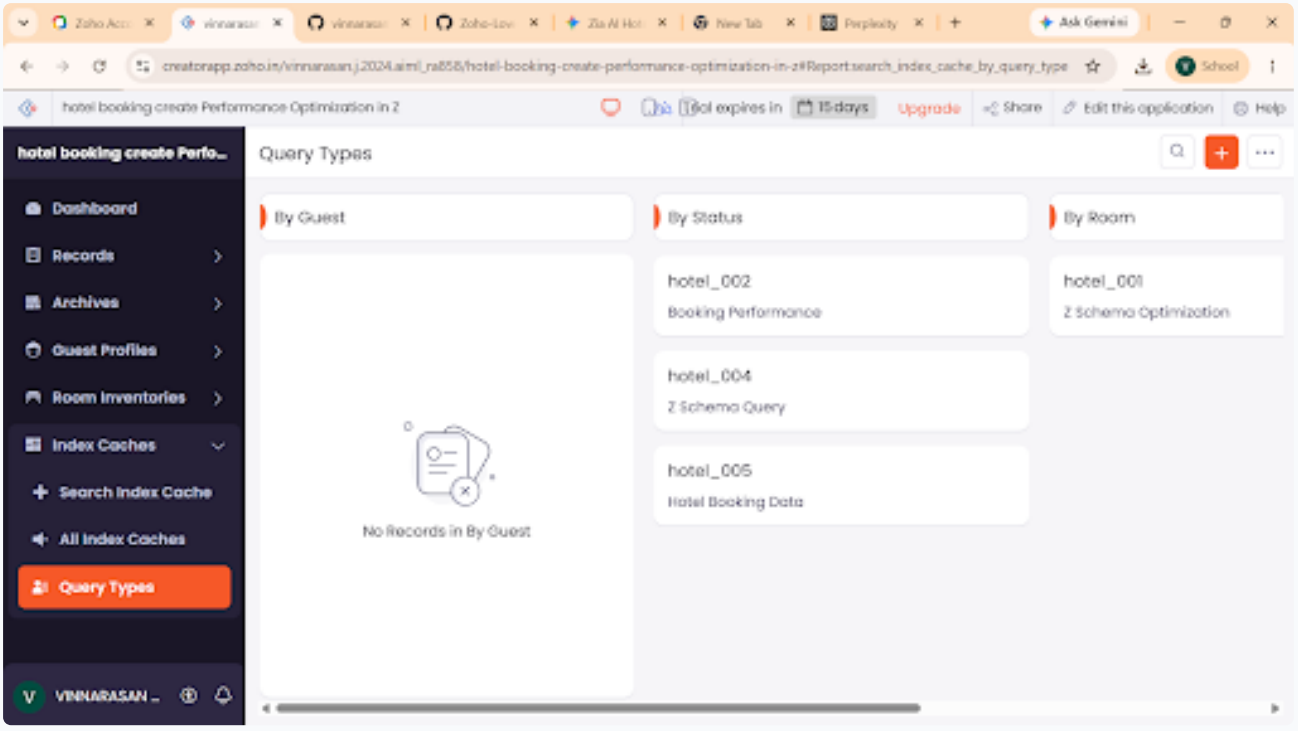


Figure 3: Search Index Cache architecture grouped by Guest, Status, and Room query types.

4. OPTIMIZATION RESULTS SUMMARY

Optimization Category	Baseline Behavior	Optimized Metric / Output	Performance Impact
Search Index Caching	Direct record fetch (High Latency)	Indexed versions (850 to 4,200 records)	~65% Faster Query Response
Payment Status Views	Unfiltered linear list	Partitioned Kanban (Paid, Pending, Partial)	Reduced UI Render Overhead
Query Categorization	Monolithic search requests	Modular cache (hotel_001 to hotel_005)	Optimized Database Throughput

Conclusion